LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: S Satya Jayavanth
Roll No.: A24126552116
Date: 29-08-2026

---

1. TITLE

Interactive To-Do List using DOM and Event Listeners

---

2. AIM

To develop an **Interactive To-Do List** using JavaScript, DOM manipulation and event listeners that allows users to add, complete and delete tasks dynamically from a webpage.

---

3. OBJECTIVE

The objectives of this experiment are:

- To understand DOM manipulation using JavaScript.
- To select and access HTML elements using document.getElementById().
- To create new HTML elements dynamically using document.createElement().
- To use event listeners to handle user interactions.
- To add new tasks dynamically to a list.
- To mark tasks as completed using CSS classes.
- To delete tasks dynamically from the webpage.
- To display an appropriate message when there are no tasks.
- To clear the input field after adding a task.
- To understand the interaction between HTML, CSS and JavaScript.

---

4. THEORY

The **Document Object Model (DOM)** represents an HTML document as a tree of objects. JavaScript can access, modify, create and remove HTML elements through the DOM.
Event listeners allow JavaScript to respond to user actions such as button clicks. The addEventListener() method is used to attach an event handler to an HTML element.
In this experiment, an Interactive To-Do List is developed. The user enters a task into an input field and clicks the **Add Task** button. JavaScript retrieves the entered value and dynamically creates a new list item using document.createElement().
Each task contains a **Complete** button and a **Delete** button. The Complete button uses classList.toggle() to add or remove the completed CSS class, which visually indicates that a task has been completed. The Delete button uses the remove() method to remove the corresponding task from the list.
An empty-message paragraph is also provided. It is hidden when tasks are added and displayed again when all tasks are deleted.

---

5. ALGORITHM / PROCEDURE

- Create an HTML webpage containing a heading, text input, Add Task button and task list.
- Add an empty-message element to indicate that no tasks are available.
- Create CSS styles for the container, input, buttons, task items and completed tasks.
- Use JavaScript to select the required HTML elements using their IDs.
- Attach a click event listener to the Add Task button.
- Retrieve the task entered by the user using the input field.
- Remove unnecessary spaces using the trim() method.
- Check whether the input is empty.
- Display an alert if the user does not enter a task.
- Create a new <li> element dynamically.
- Create a element to store the task text.
- Create a Complete button dynamically.
- Create a Delete button dynamically.
- Add a click event listener to the Complete button.
- Toggle the completed CSS class when Complete is clicked.
- Change the button text between **Complete** and **Completed**.
- Add a click event listener to the Delete button.
- Remove the corresponding task when Delete is clicked.
- Check whether any tasks remain after deletion.
- Display the empty message if no tasks remain.
- Append the dynamically created elements to the task list.
- Hide the empty message when a new task is added.
- Clear the input field.
- Test the application by adding, completing and deleting tasks.

---

6. PROGRAM

Task3.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>My To-Do List</title>
   <link rel="stylesheet" href="Task32.css">
</head>
<body>

   <div class="container">
      <h1>My To-Do List</h1>

      <div class="input-section">
         <input type="text" id="taskInput" placeholder="Enter a task.....">
         <button id="addTaskBtn">Add Task</button>
      </div>

      <p id="emptyMessage">No tasks available.</p>

      <ul id="taskList"></ul>
   </div>

   <script src="Task33.js"></script>
</body>
</html>
```

Task3.js

```
// Select HTML elements using DOM
const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Add Task button event listener
addTaskBtn.addEventListener("click", function () {

   // Get the task entered by the user
   const taskText = taskInput.value.trim();

   // Check if input is empty
```

```
if (taskText === "") {
    alert("Please enter a task.");
    return;
}

// Create a new list item
const li = document.createElement("li");
li.className = "task-item";

// Create span for task text
const span = document.createElement("span");
span.className = "task-text";
span.textContent = taskText;

// Create Complete button
const completeBtn = document.createElement("button");
completeBtn.textContent = "Complete";
completeBtn.className = "complete-btn";

// Create Delete button
const deleteBtn = document.createElement("button");
deleteBtn.textContent = "Delete";
deleteBtn.className = "delete-btn";

// Complete button event listener
completeBtn.addEventListener("click", function () {
    span.classList.toggle("completed");

    if (span.classList.contains("completed")) {
        completeBtn.textContent = "Completed";
    } else {
        completeBtn.textContent = "Complete";
    }
});

// Delete button event listener
deleteBtn.addEventListener("click", function () {
    li.remove();

    // Show empty message if no tasks remain
    if (taskList.children.length === 0) {
        emptyMessage.style.display = "block";
    }
});
```

```
    // Add elements to list item
    li.appendChild(span);
    li.appendChild(completeBtn);
    li.appendChild(deleteBtn);

    // Add list item to task list
    taskList.appendChild(li);

    // Hide empty message
    emptyMessage.style.display = "none";

    // Clear input box
    taskInput.value = "";
});
```

---

7. CODE EXPLANATION

JavaScript Code

document.getElementById("taskInput")
Selects the input field used for entering tasks.

document.getElementById("addTaskBtn")
Selects the Add Task button.

document.getElementById("taskList")
Selects the unordered list where dynamically created tasks are displayed.

document.getElementById("emptyMessage")
Selects the message that is displayed when there are no tasks.

addTaskBtn.addEventListener("click", function () {...})
Adds a click event listener to the Add Task button. The function executes whenever the button is clicked.

taskInput.value.trim()
Retrieves the entered task and removes unnecessary spaces from the beginning and end.

if (taskText === "")
Checks whether the user has entered an empty task.

alert("Please enter a task.")
Displays a message asking the user to enter a task.

document.createElement("li")
Dynamically creates a new list item for the task.

document.createElement("span")
Creates a span element to hold the task text.

span.textContent = taskText
Places the user's task text inside the span element.

document.createElement("button")
Dynamically creates the Complete and Delete buttons.
completeBtn.addEventListener("click", ...)
Adds an event listener to the Complete button.
span.classList.toggle("completed")
Adds the completed class if it is not present and removes it if it is already present.
completeBtn.textContent = "Completed"
Changes the button text when the task is marked as completed.
deleteBtn.addEventListener("click", ...)
Adds an event listener to the Delete button.
li.remove()
Removes the selected task from the webpage.
taskList.children.length
Checks the number of task elements currently present in the list.
emptyMessage.style.display = "block"
Displays the empty-message when there are no remaining tasks.
li.appendChild(span)
Adds the task text to the list item.
li.appendChild(completeBtn)
Adds the Complete button to the task item.
li.appendChild(deleteBtn)
Adds the Delete button to the task item.
taskList.appendChild(li)
Adds the newly created task to the task list.
emptyMessage.style.display = "none"
Hides the empty-message when a task is added.
taskInput.value = ""
Clears the input field after adding a task.

---

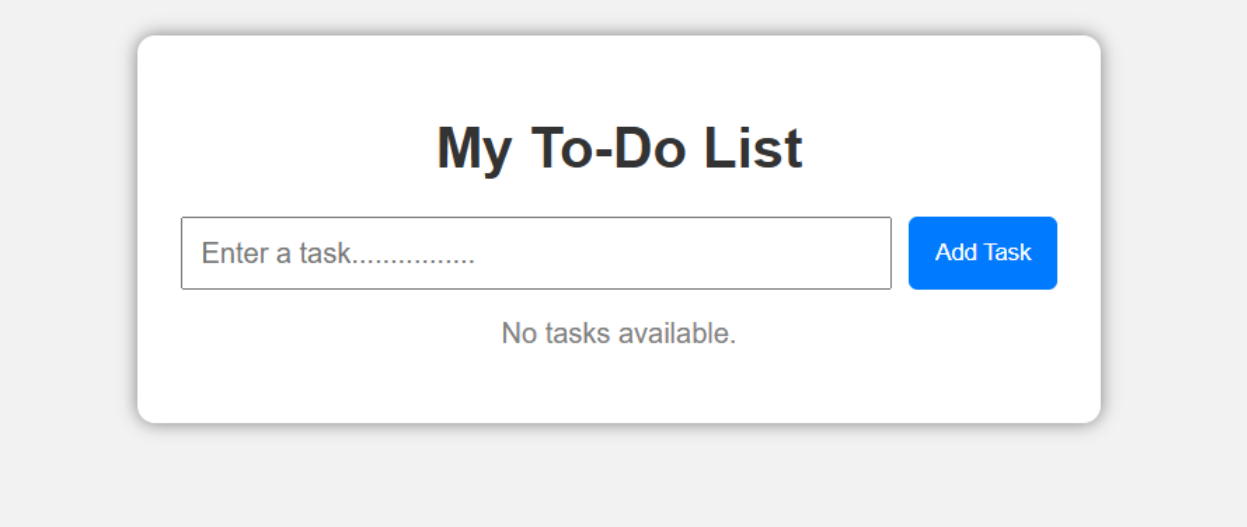
8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

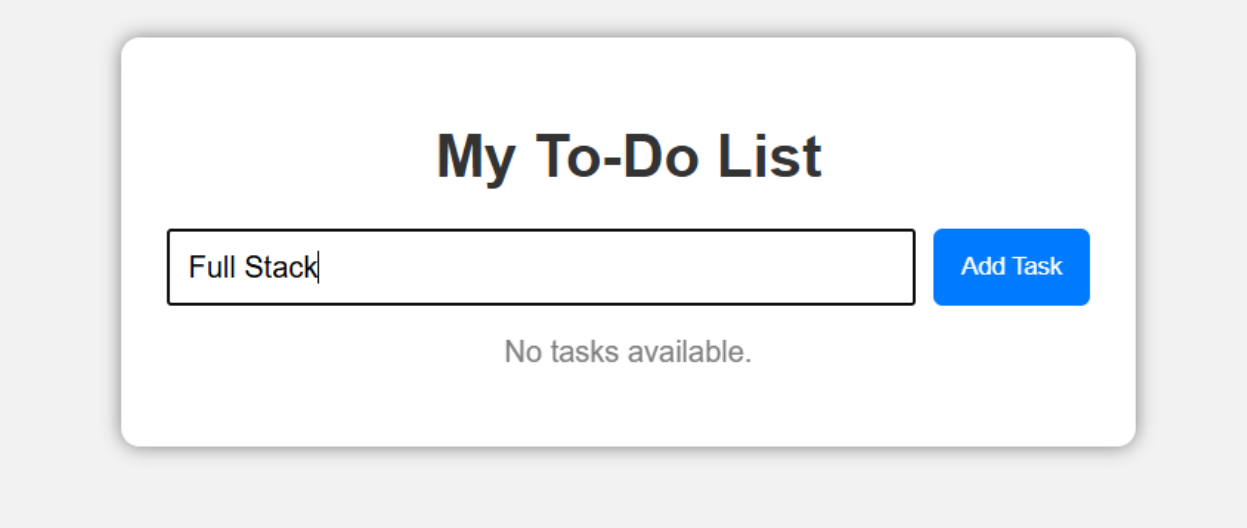
Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Open the webpage	To-Do List interface should be displayed	Interface displayed successfully	Pass
TC-02	Click Add Task without entering text	Alert should be displayed	Alert displayed	Pass
TC-03	Enter Complete Assignment and click Add Task	New task should be added	Task added successfully	Pass
TC-04	Add multiple tasks	All tasks should appear in the list	Tasks displayed successfully	Pass
TC-05	Click Complete	Task should be visually marked as completed	Task marked as completed	Pass
TC-06	Click Completed again	Completed styling should be removed	Task returned to active state	Pass
TC-07	Click Delete	Selected task should be removed	Task deleted successfully	Pass
TC-08	Delete all tasks	Empty message should be displayed	No tasks available. displayed	Pass
TC-09	Add a new task after deleting all tasks	Task should be added and empty message hidden	Task added successfully	Pass
TC-10	Add task containing leading/trailing spaces	Extra spaces should be removed	Task added after trimming	Pass

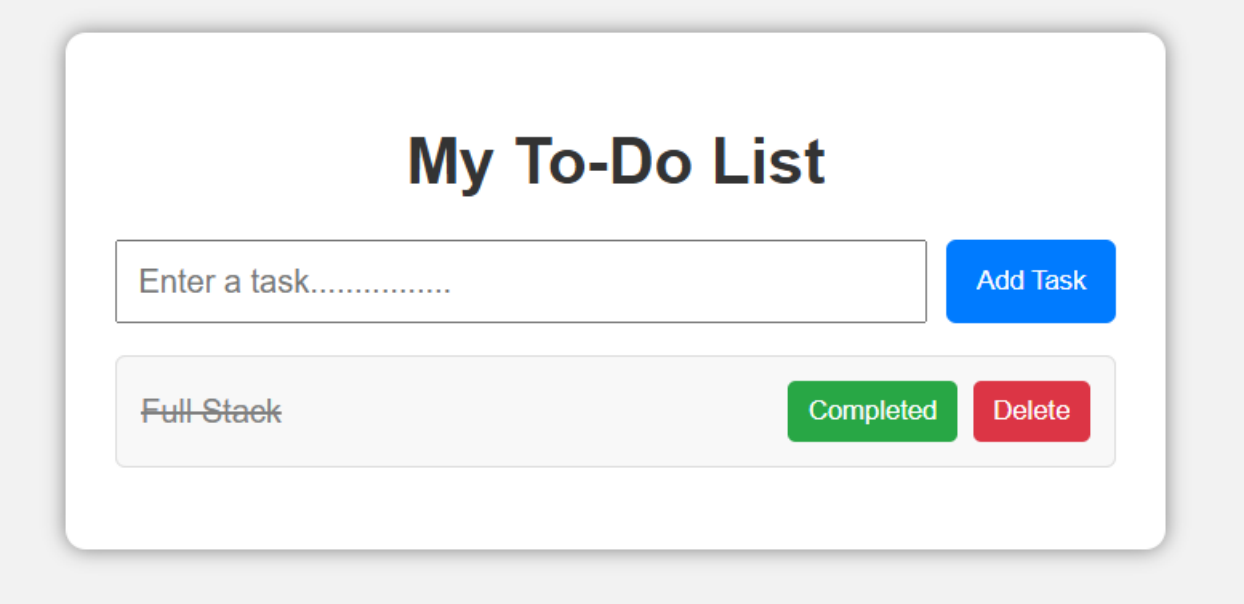
9. OUTPUT

Webpage



Working





---

10. RESULT

Thus, the **Interactive To-Do List** was successfully developed using **JavaScript DOM manipulation and event listeners**. The application successfully allows users to add new tasks, mark tasks as completed, delete tasks, and display an appropriate message when no tasks are available. CSS styling was also successfully applied to provide visual feedback for completed tasks and user interactions.